

Schema Changes Reporting

1 Overview

The goal of is to provide automated reporting of JSON schema changes between platform releases across FOLIO modules. Schemas define API contracts for stored entities and API requests/responses. Timely detection of schema modifications helps prevent API incompatibility. The main scenarios of schema changes include:

- Adding a new field
- Changing the name or type of a field
- Removing a field
- Moving a field (changing its structure/position)
- Adding or removing a field from the required list

The solution to track schema changes can be implemented on two levels:

- **Module-level workflow** — runs inside each module repository, compares schemas between two tags or commits, produces a diff report.
- **Central workflow** — resolves module versions from platform releases, calls module workflows in parallel, and merges individual reports into a combined artifact.

2 Functional Requirements

Functional requirements for module-level and central workflows are described below.

2.1 Module-level workflow

1. Compare JSON schemas in the `ramls/` directory between two versions of a module, recursively including subdirectories and excluding `ramls/examples/` and `ramls/raml-util/`.
2. Trigger automatically on schema changes in push and pull request, on release published, on manual dispatch, and when called from the central workflow.
3. Build a report containing a changed files table and a normalized diff (JSON keys sorted) for each file, with a raw textual diff as fallback.
4. Post the report as a sticky comment on pull requests (updated on subsequent pushes, not duplicated).
5. Upload the report in Markdown and HTML formats as a workflow artifact.
6. Correctly determine the previous version using semantic versioning during automatic runs.
7. Handle edge cases: first push to a new branch, single tag in repository, invalid refs, tags with identical dates.

2.2 Central workflow

1. Automatically determine the two most recent `platform-lsp` releases by tag creation date.
2. For each release, walk the chain `install-applications.json` → app template → module version to identify which module versions are included.
3. Call each module's workflow in parallel with the resolved versions.
4. Collect individual reports into a single combined HTML file.
5. Provide an option to specify release tags manually instead of auto-detecting. If only one tag is provided, fail with a clear error.
6. Validate that manually specified tags exist in the repository. If not found, fail with an error listing available tags.
7. Log all intermediate steps: tags with dates, applications, modules in templates (target modules marked), and a final version summary table.

3 Approach

Schemas are stored as JSON files in the `ramls/` directory of each module (e.g. `mod-inventory-storage`, `mod-users`). Changes in schemas are tracked by comparing these files between two Git refs using GitHub Actions.

3.1 Module-level diff approach

Each module workflow compares JSON schema files between two Git refs (tags or commits) within the module repository. The approach consists of the following steps:

1. Depending on the trigger event, the workflow resolves which two points in Git history to compare:
 - On push — the previous commit (`before`) and the current commit.
 - On pull request — the PR base branch SHA and the PR head SHA.
 - On release — the current release tag and the previous tag by semantic version.
 - On manual or external call — explicitly provided refs, or the two latest tags if none specified.
2. A `git diff` with rename detection is executed between the two refs, scoped to `ramls/**/*.json` and excluding `ramls/examples/` and `ramls/raml-util/`. The output is a list of changed files with their status (Modified, Added, Deleted, Renamed).
3. For each changed file, both versions are extracted and normalized by sorting JSON keys alphabetically (`jq -S .`). A unified diff is then computed on the normalized output. This eliminates false positives caused by key reordering or formatting differences.

4. If the normalized diff shows no structural changes (e.g. only whitespace or formatting differences), a raw textual diff is shown instead.
5. The results are assembled into a Markdown report containing a summary table of changed files and per-file diffs. The Markdown is then converted to a standalone HTML file with embedded styles using Pandoc.

3.2 Central workflow: module version resolution

The central workflow determines at the first stage which module versions belong to each platform release by walking the following chain:

1. Read release tags from `platform-lsp` repository sorted by creation date.
2. For each release tag, read `install-applications.json`.
3. For each application entry, fetch the corresponding `app-*.template.json` from the application repository.
4. Extract target module names and versions from the template's `modules[]` array.

This produces a mapping: *release tag* $\rightarrow$ *application* $\rightarrow$ *module version*, which is then passed to module workflows as base/head tags.

3.3 Execution flow

The central workflow executes in the following order:

1. **resolve_tags** - clones `platform-lsp`, determines two release tags (auto-detected or manually specified), resolves module versions through application templates, outputs base/head tags for each module.
2. **call-mod-*** -calls each module's reporting workflow in parallel with the resolved tags.
3. **collect-reports** - downloads individual HTML reports from all modules, merges them into a single combined HTML artifact.

```

                                +--> mod-inventory-storage --+
resolve_tags -----+                                +--> collect-reports
                                +--> mod-* -----+

```

4 Application Scenarios

4.1 Module-level report

4.1.1 Pull request report

When a pull request modifies schema files, the module workflow generates a diff report and posts it as an auto-generated sticky comment in the pull request (Fig. 1). The comment is updated on each subsequent push to the same PR.

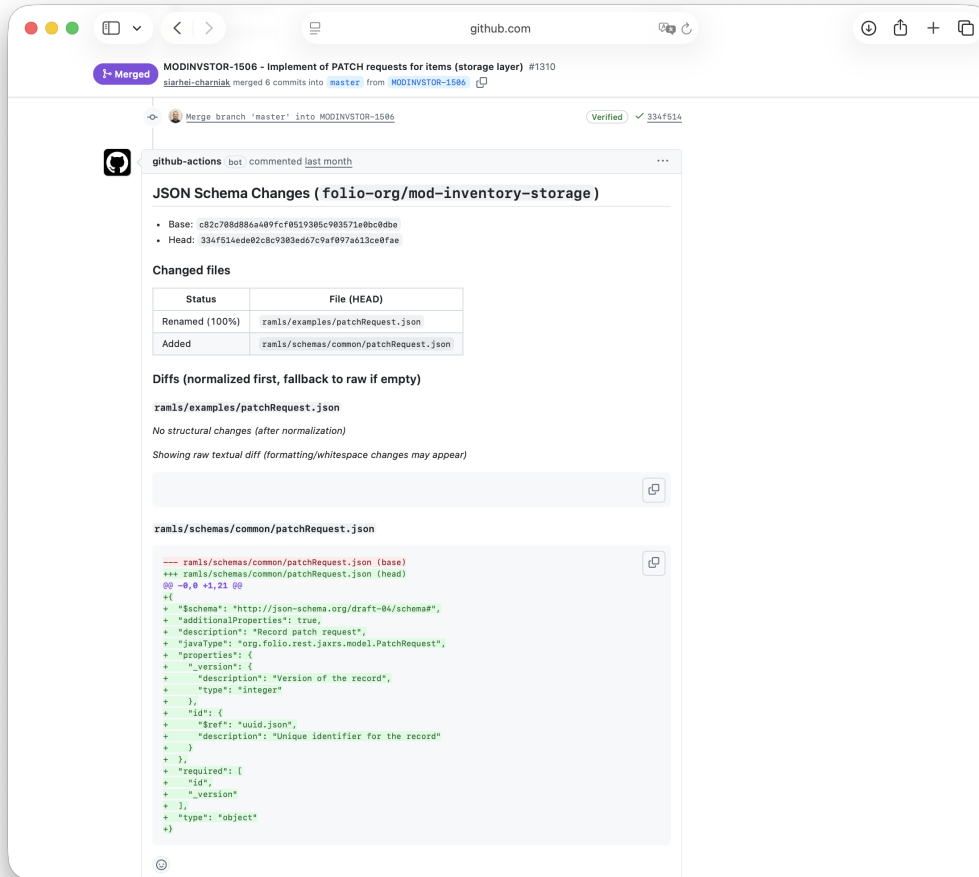


Figure 1: Auto-generated PR comment with schema change report.

4.1.2 Commit report

When a commit with schema changes is pushed, the module workflow generates a report available as a GitHub Actions artifact (Fig. 2).

4.1.3 Report between release tags

A user can manually trigger the module workflow and specify base/head tags to compare arbitrary versions from the GitHub Actions menu (Fig. 3). The generated report is available as an artifact (Fig. 4).

4.2 Cross-module-report

4.2.1 Report with auto-detected releases

A user can trigger the central workflow without specifying any inputs (Fig. 5). The workflow automatically determines the two most recent release tags in `platform-lsp` by creation date, resolves module versions from each release, and generates a combined HTML report containing diffs from all tracked modules as a single downloadable artifact.

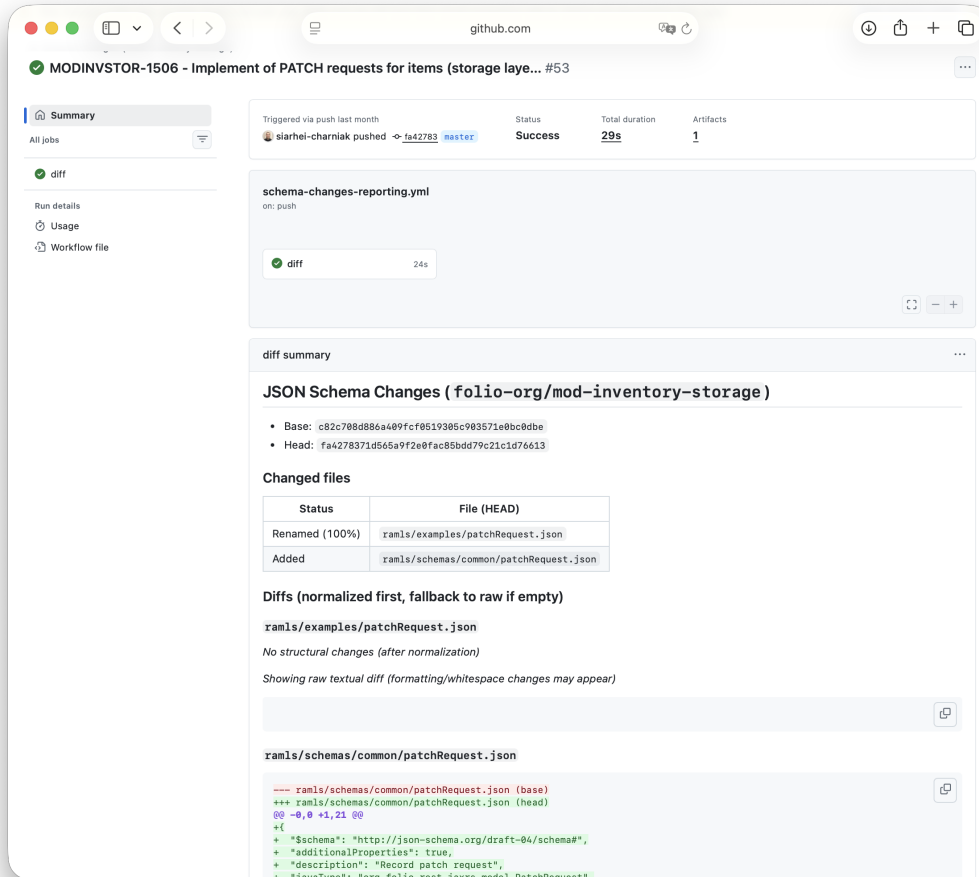


Figure 2: Auto-generated Actions report for a commit with schema changes.

4.2.2 Report with manually specified releases

A user can trigger the central workflow and specify base and head release tags manually to compare any two platform releases (Fig. 6). Both tags must be provided - partial input (only one tag) results in an error. If a specified tag does not exist, the workflow fails with a message listing available tags. The resulting combined report is identical in format to the auto-detected scenario.

5 Report Content

Each module report contains:

1. **Header** - repository name, base ref, head ref.
2. **Changed files table** - file status (Modified, Added, Deleted, Renamed) and file path.
3. **Per-file diff** - normalized structural diff with JSON keys sorted alphabetically. If normalization shows no changes, a raw textual diff is shown as fallback. Output is truncated at 500 lines with a notice.

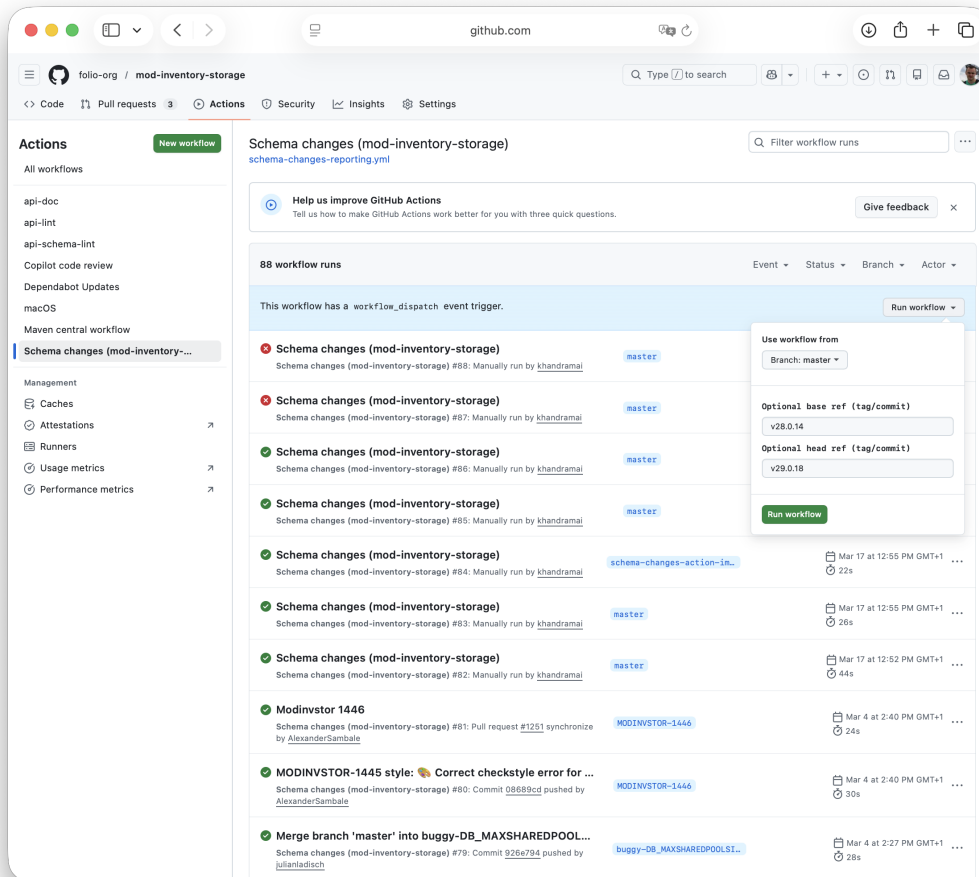


Figure 3: Manual trigger for generating a report between tags.

Reports are generated in two formats: Markdown (used for PR comments and raw viewing) and standalone HTML with GitHub-like styling (used for the combined report and direct viewing).

Advantages

- Built-in GitHub mechanisms - no separate applications to develop or maintain.
- Event-driven - reports are generated automatically on push, pull request, and release events without scheduled jobs.
- Centralized cross-module reporting - a single workflow compares all tracked modules between platform releases.
- Parallel execution - module reports are generated simultaneously, reducing total execution time.
- Manual override - release tags can be specified manually for ad-hoc comparisons.

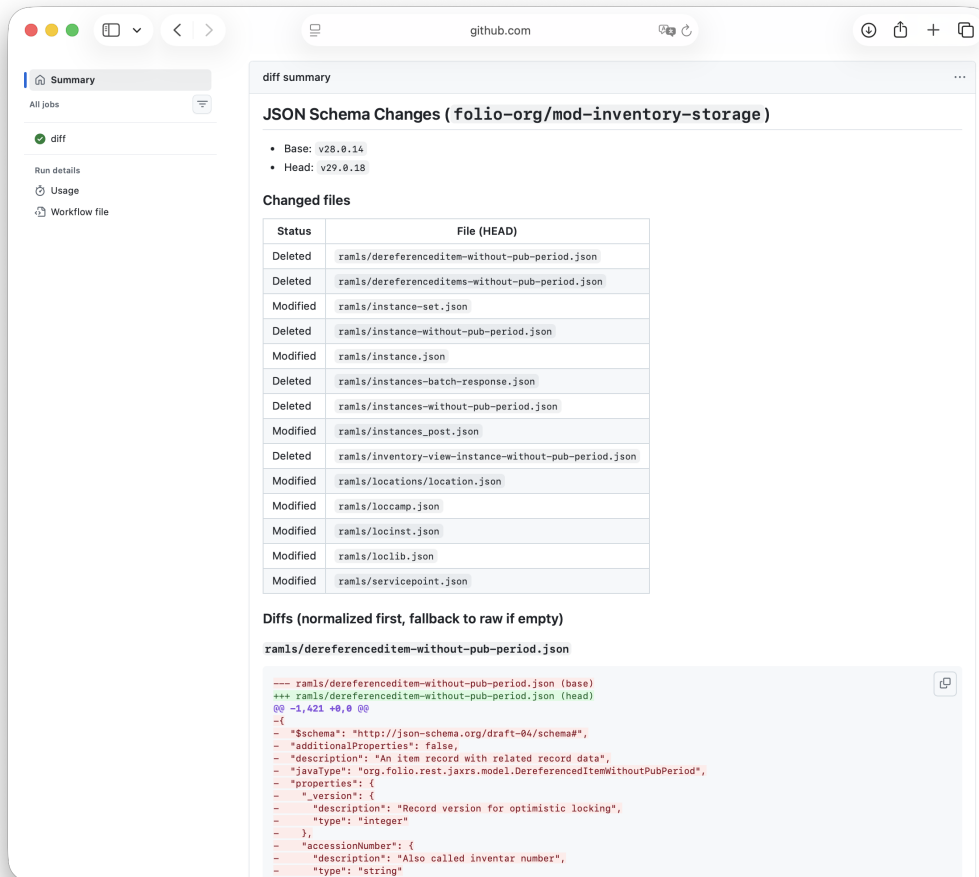


Figure 4: Report generated based on tag information.

Limitations

- A GitHub Action workflow file must be added to each module where schema tracking is needed.
- Adding a new module to central reporting requires manual configuration (adding to target list, outputs, and job definition).
- Tool currently detects changes but does not perform breaking change analysis or schema validation.

References

- PoC repository: <https://github.com/folio-org/schema-changes-reporting>
- Central workflow: <https://github.com/folio-org/schema-changes-reporting/blob/master/.github/workflows/central-reporting.yml>
- Module workflow (for example, mod-users): <https://github.com/folio-org/mod-users/blob/master/.github/workflows/schema-changes-reporting.yml>

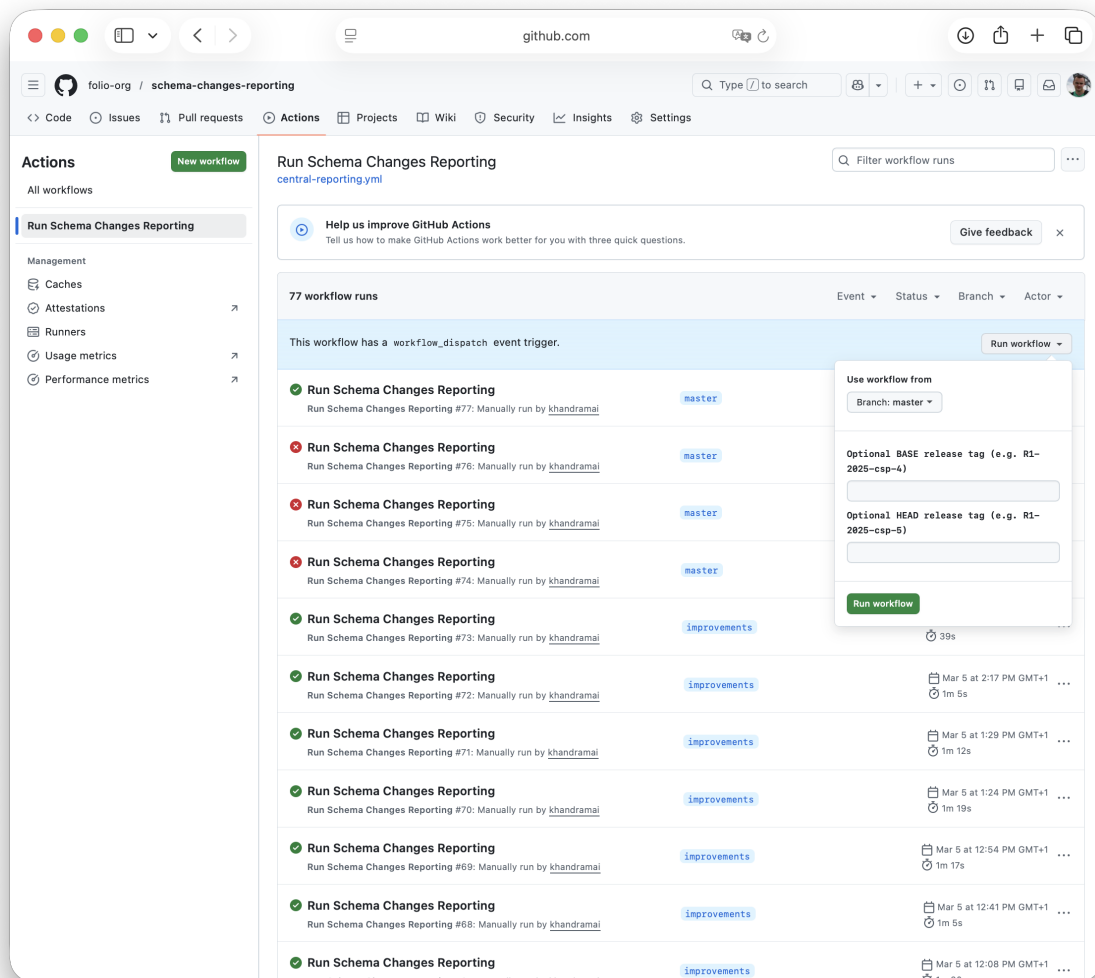


Figure 5: The central workflow without specifying any inputs (two most recent release).

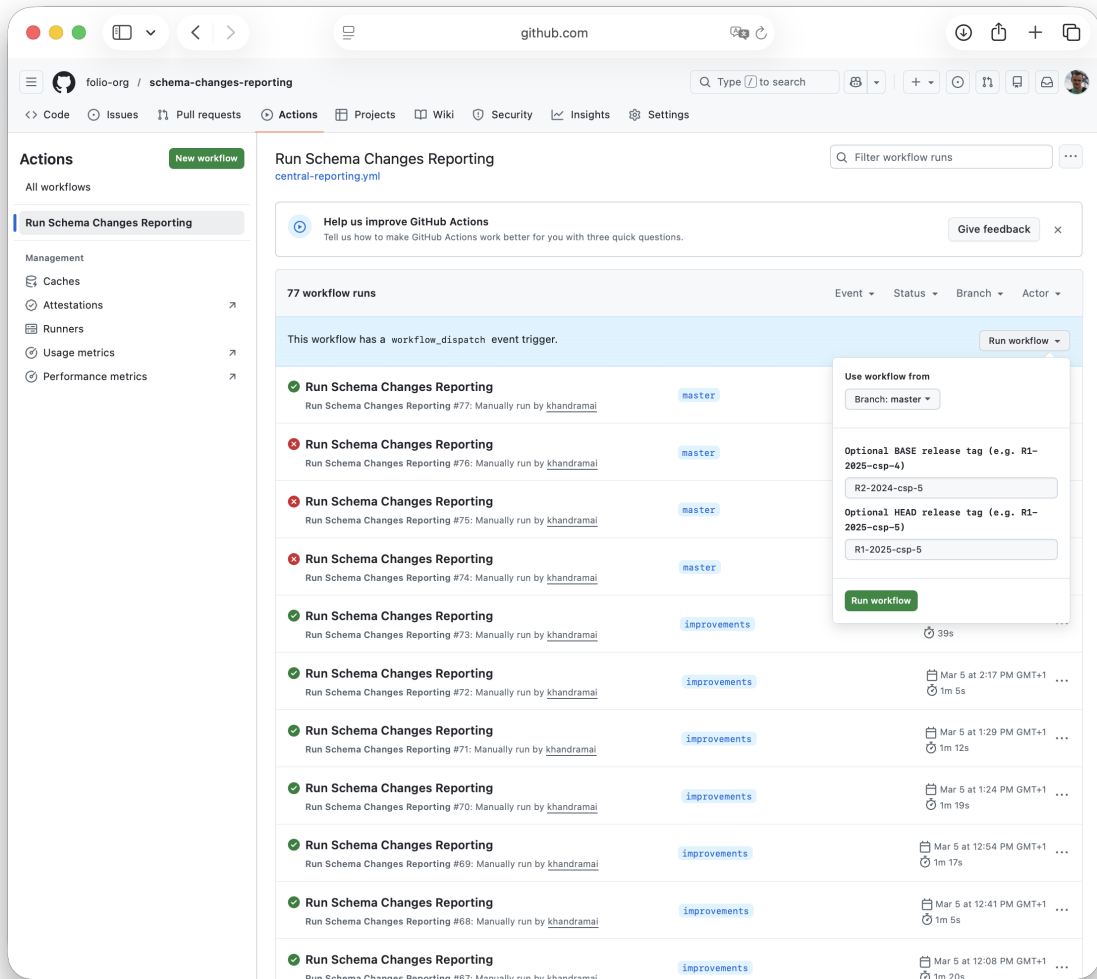


Figure 6: The central workflow with specifying LSP releases.